

PROJECT BRIEFING

1. Project Introduction

- This project, titled "AI-Powered Face Recognition Attendance & Security System," is an advanced IoT solution designed to automate identity verification. At its core, the system utilizes the ESP32-CAM—a low-cost, high-performance microcontroller with an integrated camera—to capture and transmit facial data wirelessly. By combining the power of Embedded Systems with Computer Vision, we have developed a robust platform that not only marks attendance but also serves as a security gateway, ensuring that only authorized personnel are logged into the system.

2. Problem Statement & Motivation

- Traditional biometric systems, such as fingerprint scanners, have long been the industry standard. However, they possess a significant drawback: hygiene. In a post-pandemic world, a system that requires hundreds of users to touch the same sensor surface daily is considered unhygienic and a potential vector for spreading germs and viruses. Furthermore, physical sensors are prone to wear and tear over time.
- Our project addresses these issues by implementing a Zero-Touch (Contactless) solution. By using facial recognition, we provide a safer, more sanitary, and more efficient alternative that maintains high security without any physical contact between the user and the hardware.

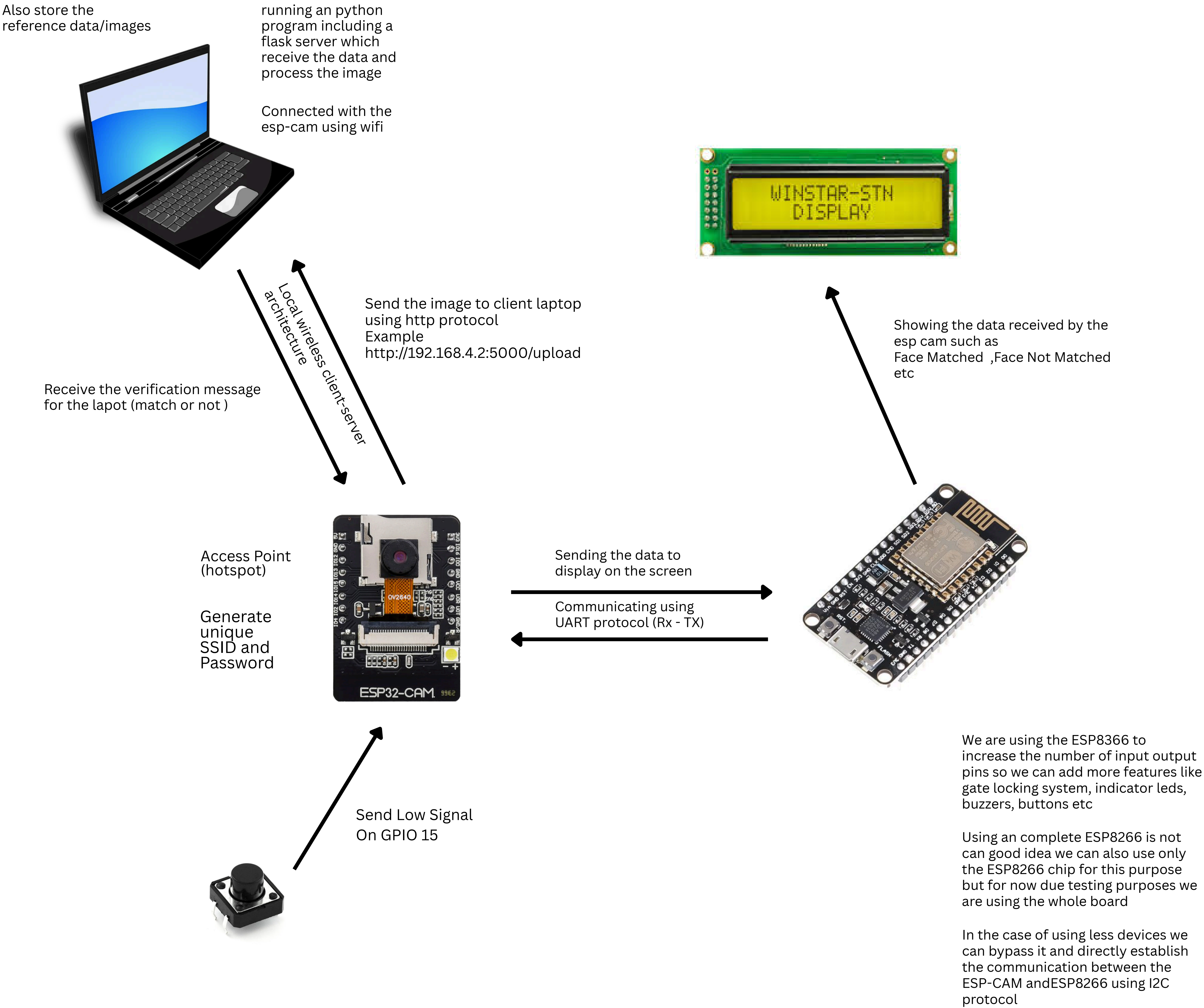
3. System Architecture & Working Principle

- The operation of the system is divided into three synchronized phases:
- **Wireless Communication:** The ESP32-CAM is programmed to operate as a wireless node. It captures a high-resolution image whenever a person stands before the camera and transmits this data over a local network to a central processing unit (a laptop or server) running a Python-based Flask server.
- **Facial Analysis:** Once the Python backend receives the image, it utilizes the Face Recognition library (built on dlib's Deep Learning models). The system extracts unique landmarks from the face and compares them against a pre-authorized database of "Known Faces."
- **Real-time Processing & Feedback:**
- **If a match is found:** The system identifies the individual, logs their name and exact timestamp into a CSV-based digital register, and displays a "Welcome" message on the Tkinter GUI.
- **If no match is found:** The system flags an "Access Denied" alert, providing an extra layer of security against unauthorized entry.

4. Key Advantages

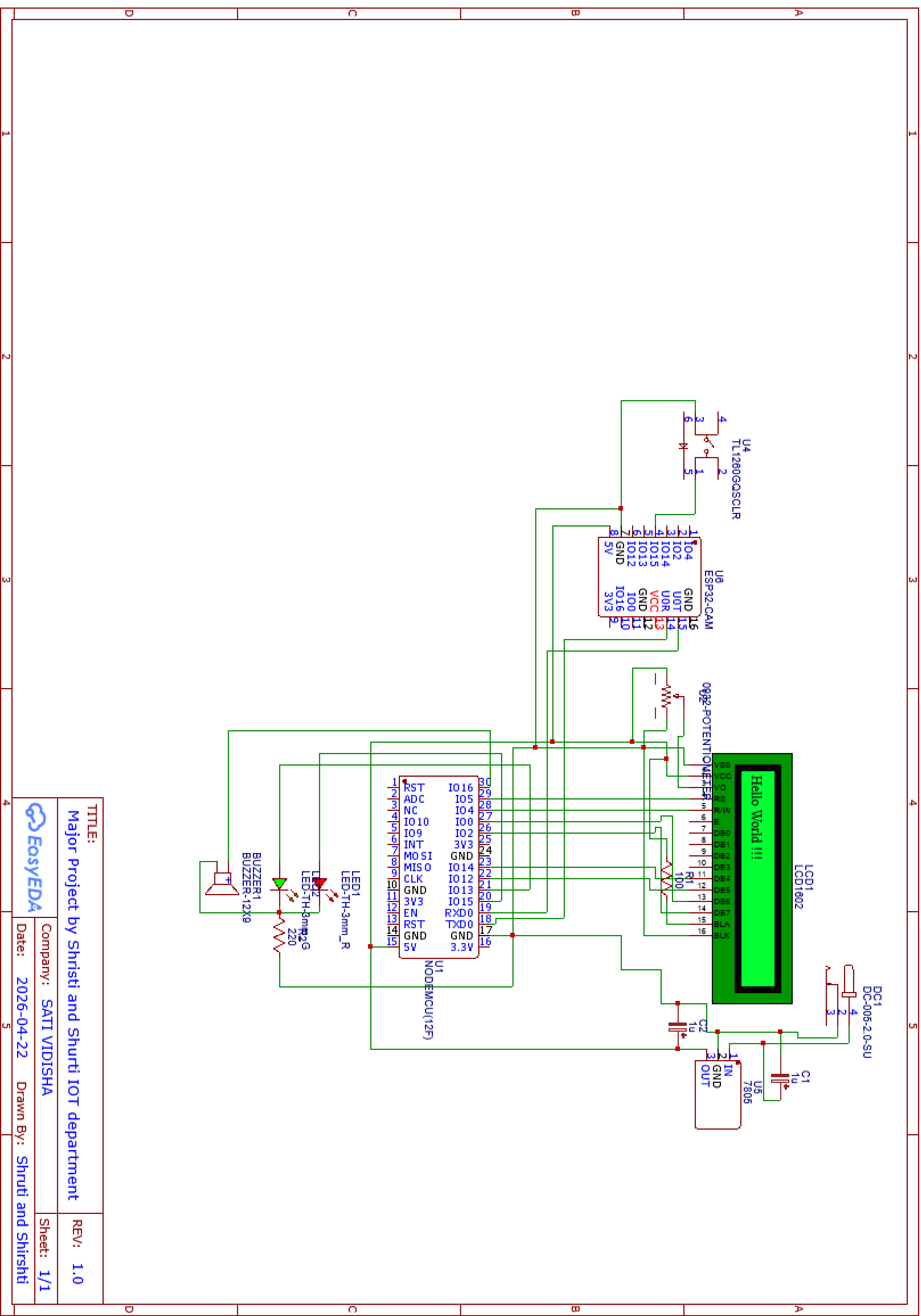
- **Contactless & Hygienic:** Eliminates the need for physical contact, making it ideal for schools, offices, and hospitals.
- **Wireless Flexibility:** The use of an ESP32-CAM allows the hardware to be mounted anywhere within Wi-Fi range without bulky data cables.
- **Automated Record Keeping:** Replaces manual registers with a tamper-proof digital log, reducing administrative workload.

OVERVIEW OF CAM BASED ATTENDANCE SYSTEM (HARDWARE)

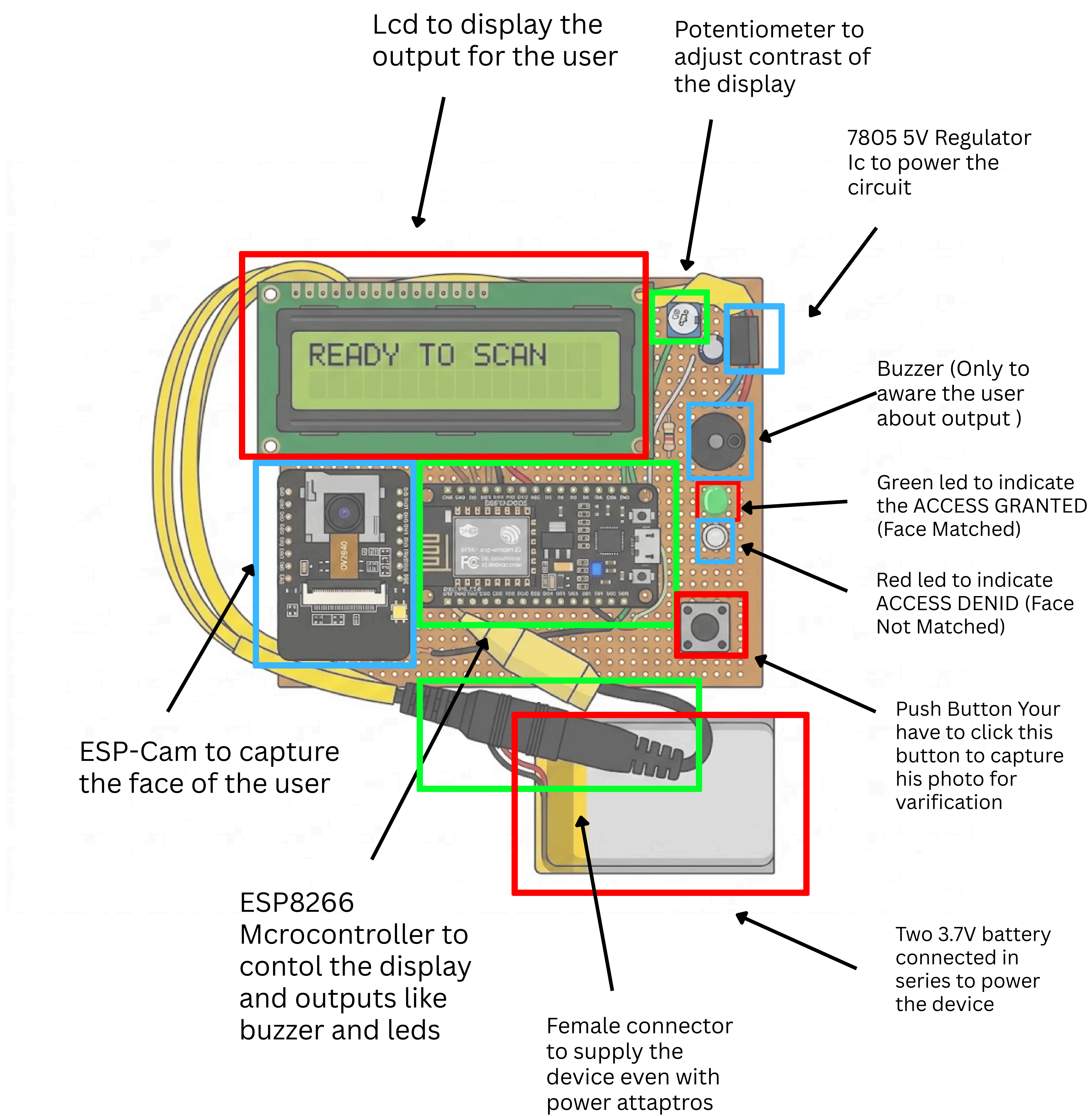


Communication Establishment Summary

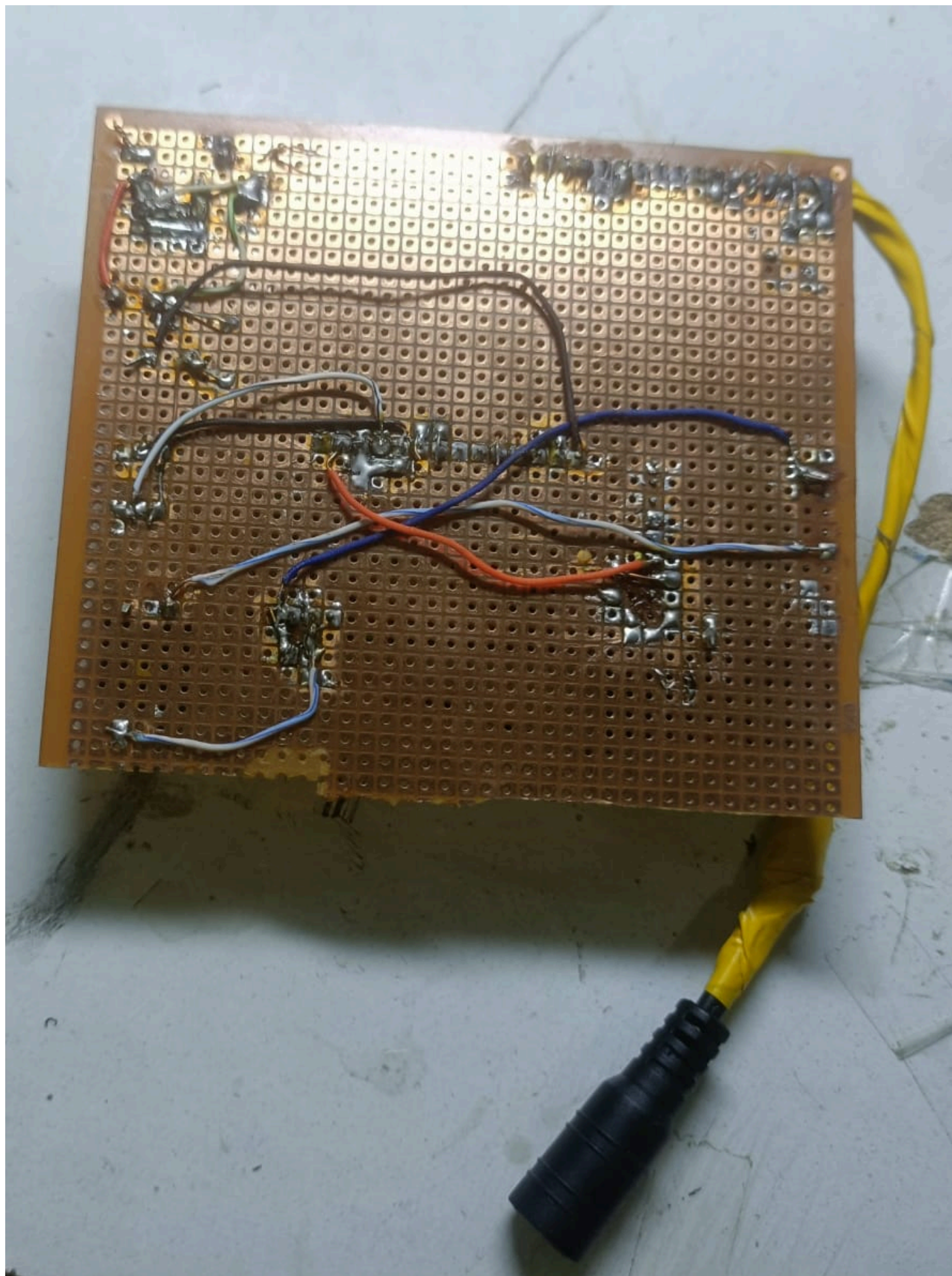
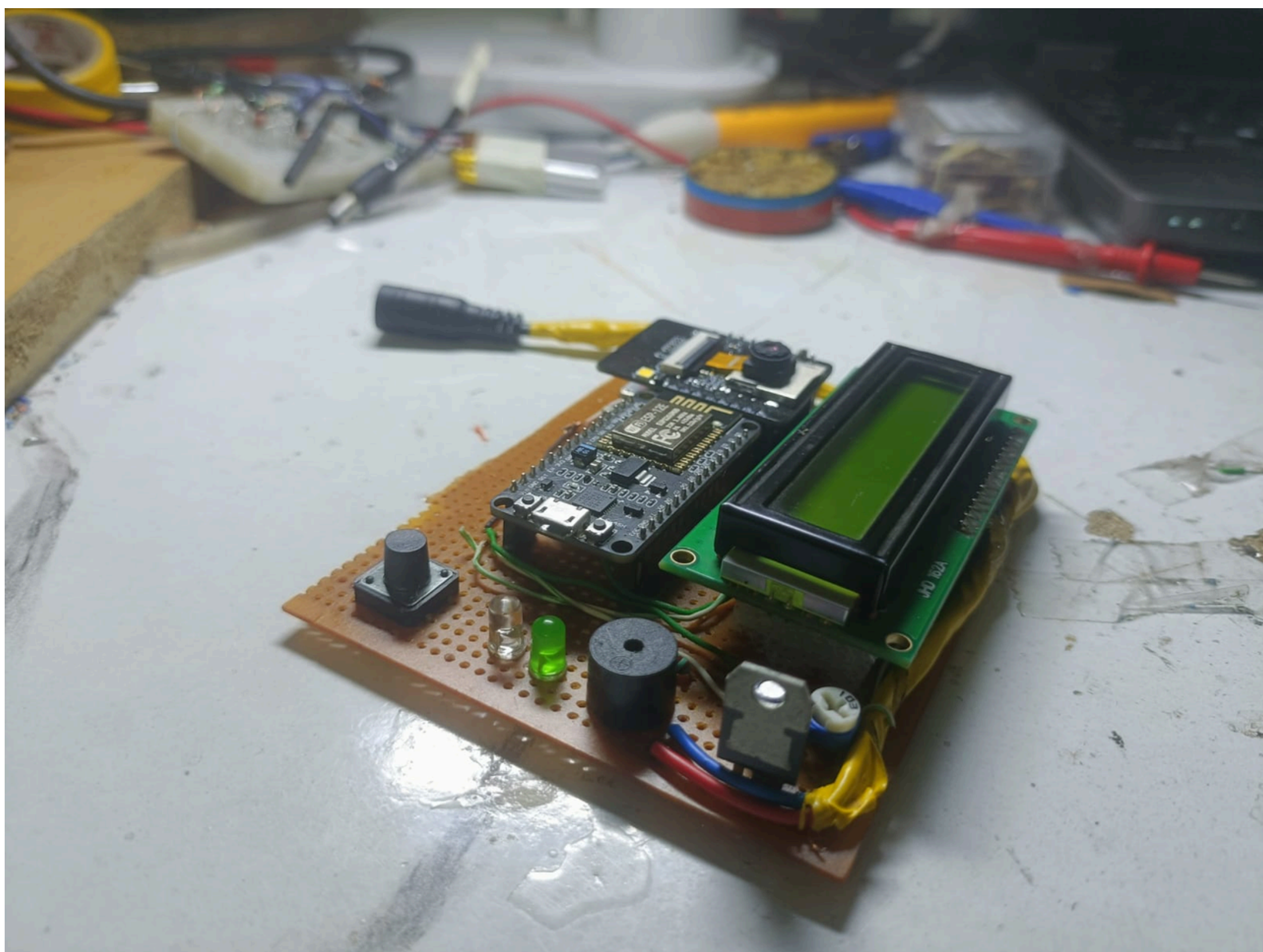
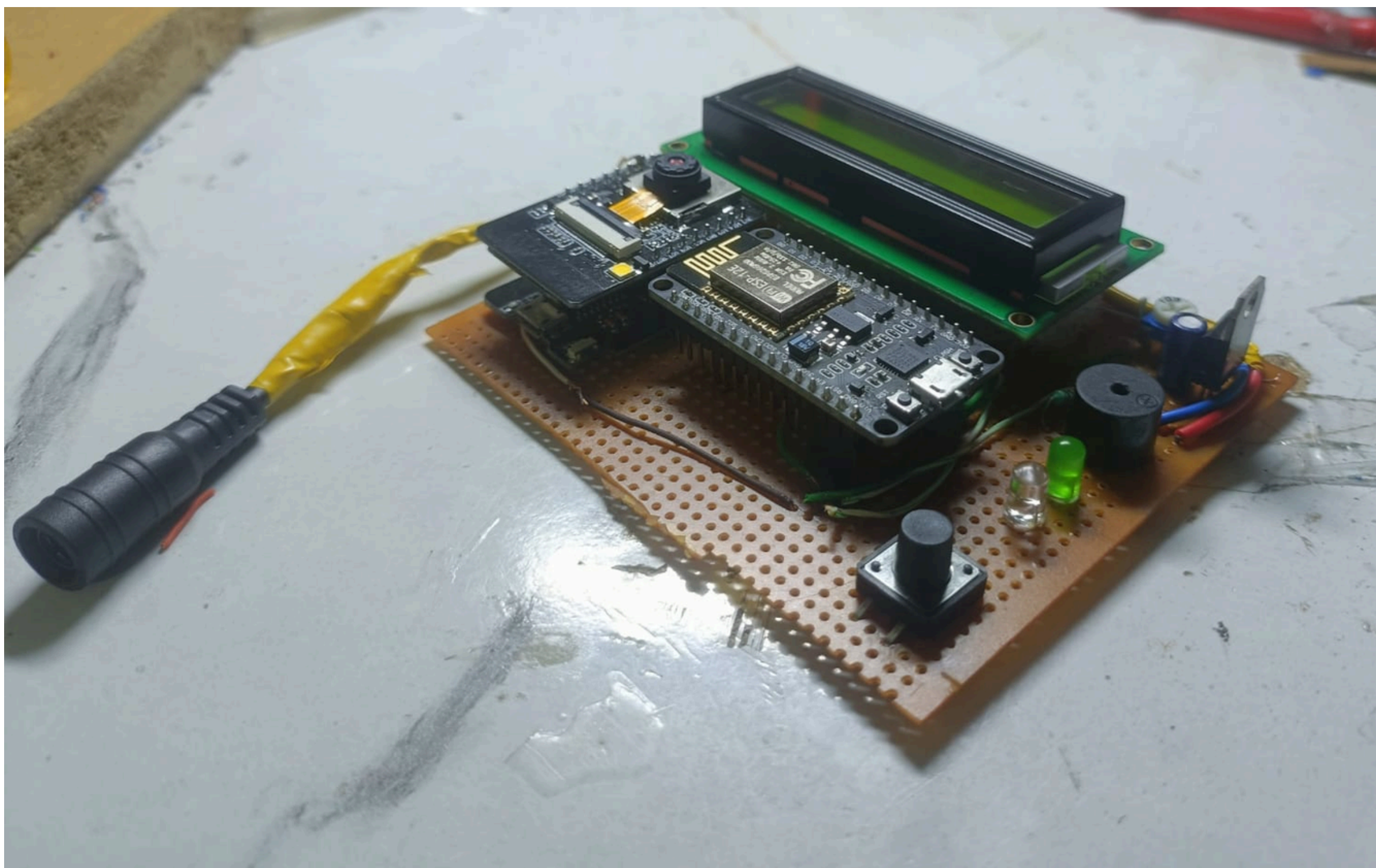
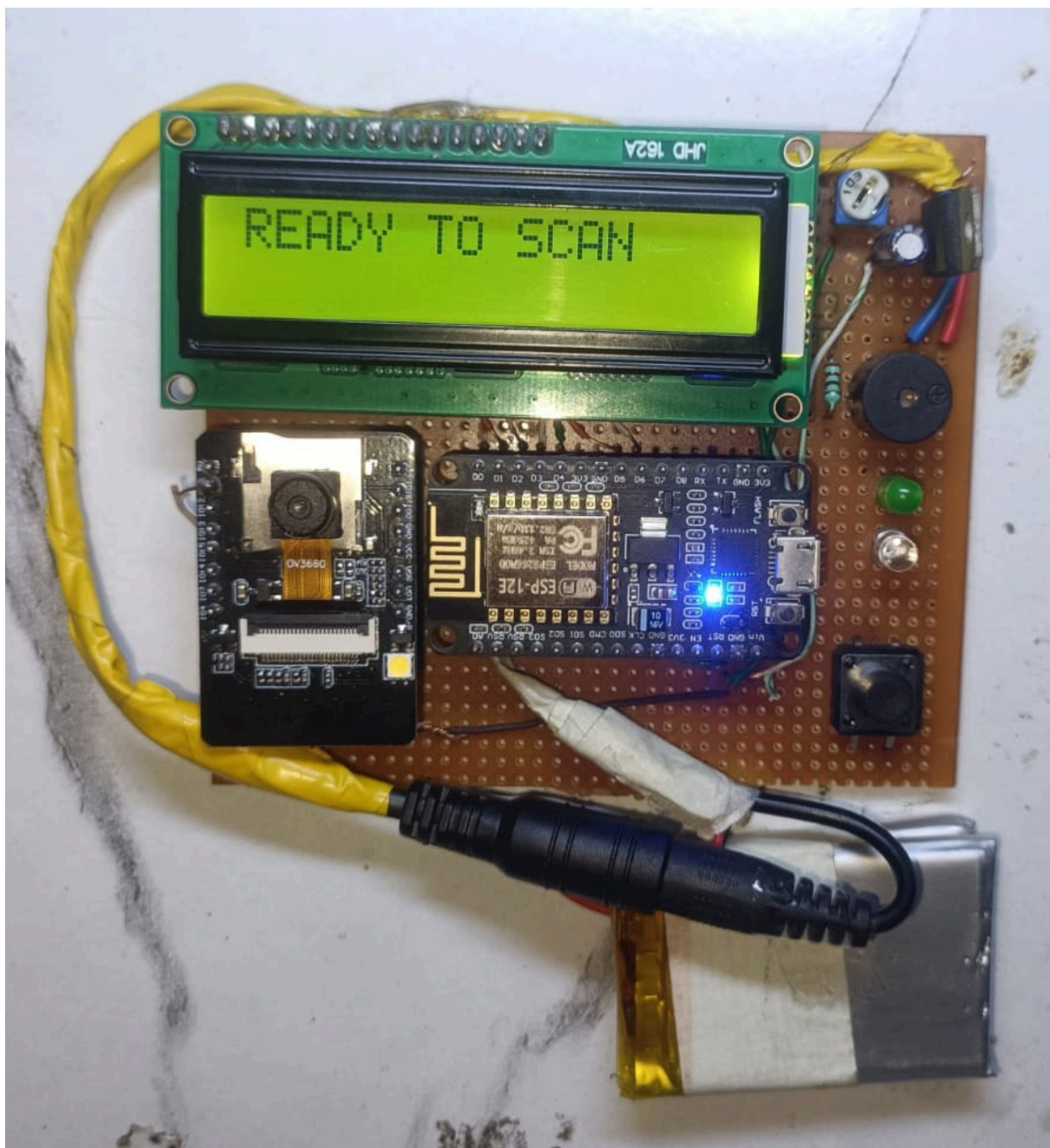
Stage	Data Direction	Protocol	Purpose
Connection	Laptop $\longleftrightarrow$ ESP32-CAM	Wi-Fi (802.11)	Establish a local network (No Internet).
Upload	ESP32-CAM $\rightarrow$ Laptop	HTTP POST	Transfer the JPEG image for processing.
Recognition	Internal Laptop	Python/OpenCV	Identify the person in the photo.
Result	Laptop $\rightarrow$ ESP32-CAM	HTTP Response	Send the match status/name back.
Output	ESP32-CAM $\rightarrow$ ESP8266	Serial (UART)	Bridge the data to the LCD screen.



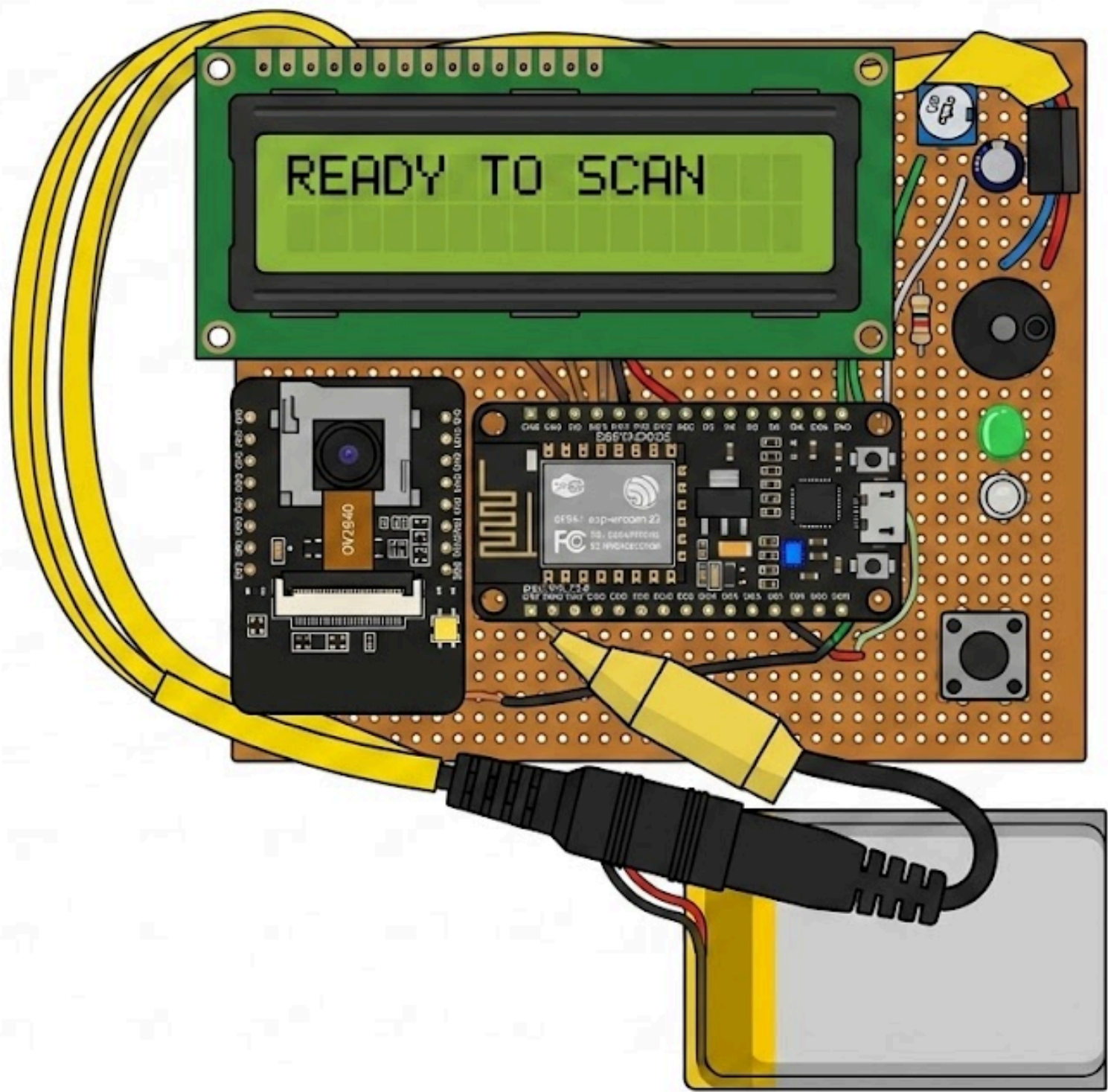
Device Description



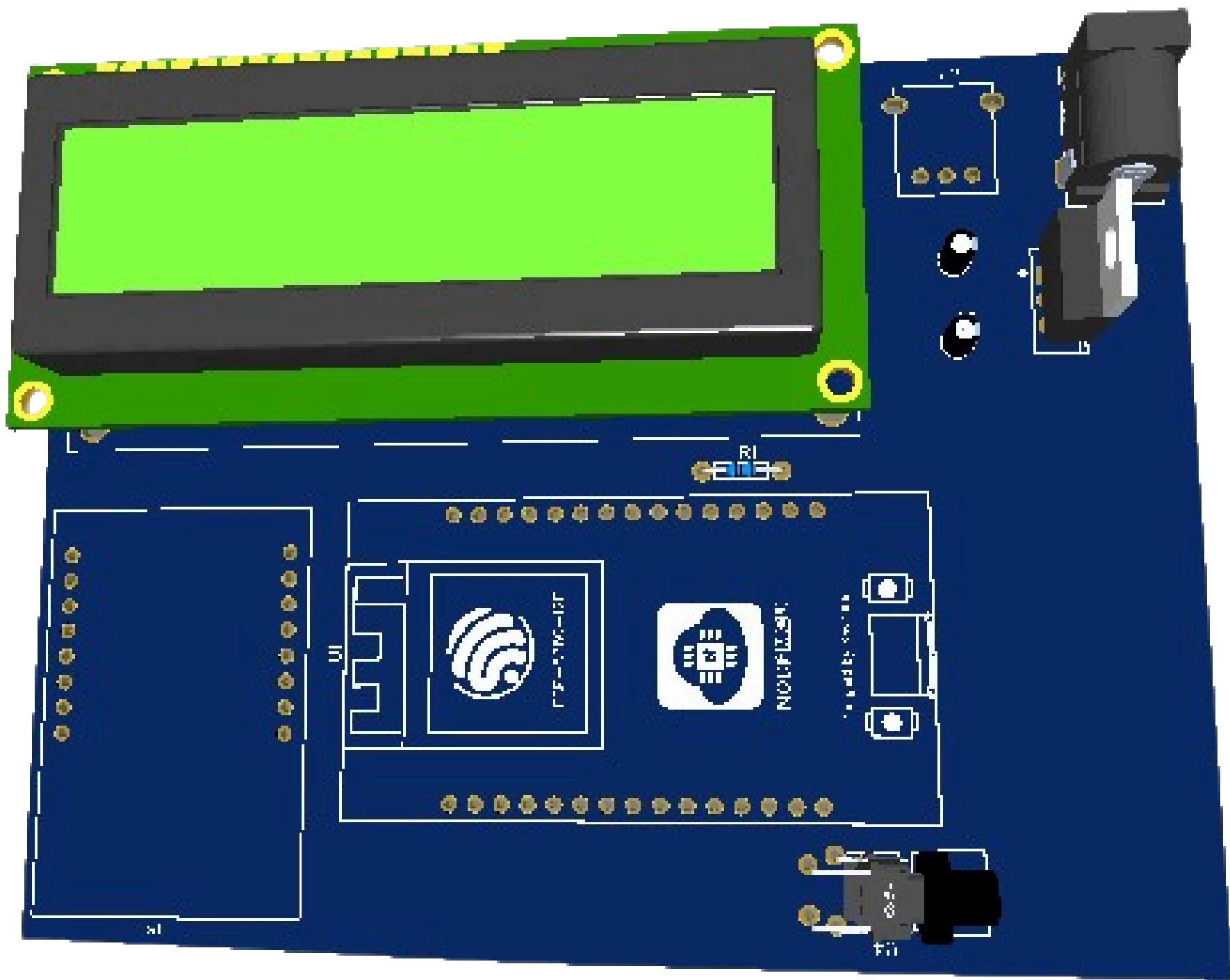
Project Images



Real Images of Prototype



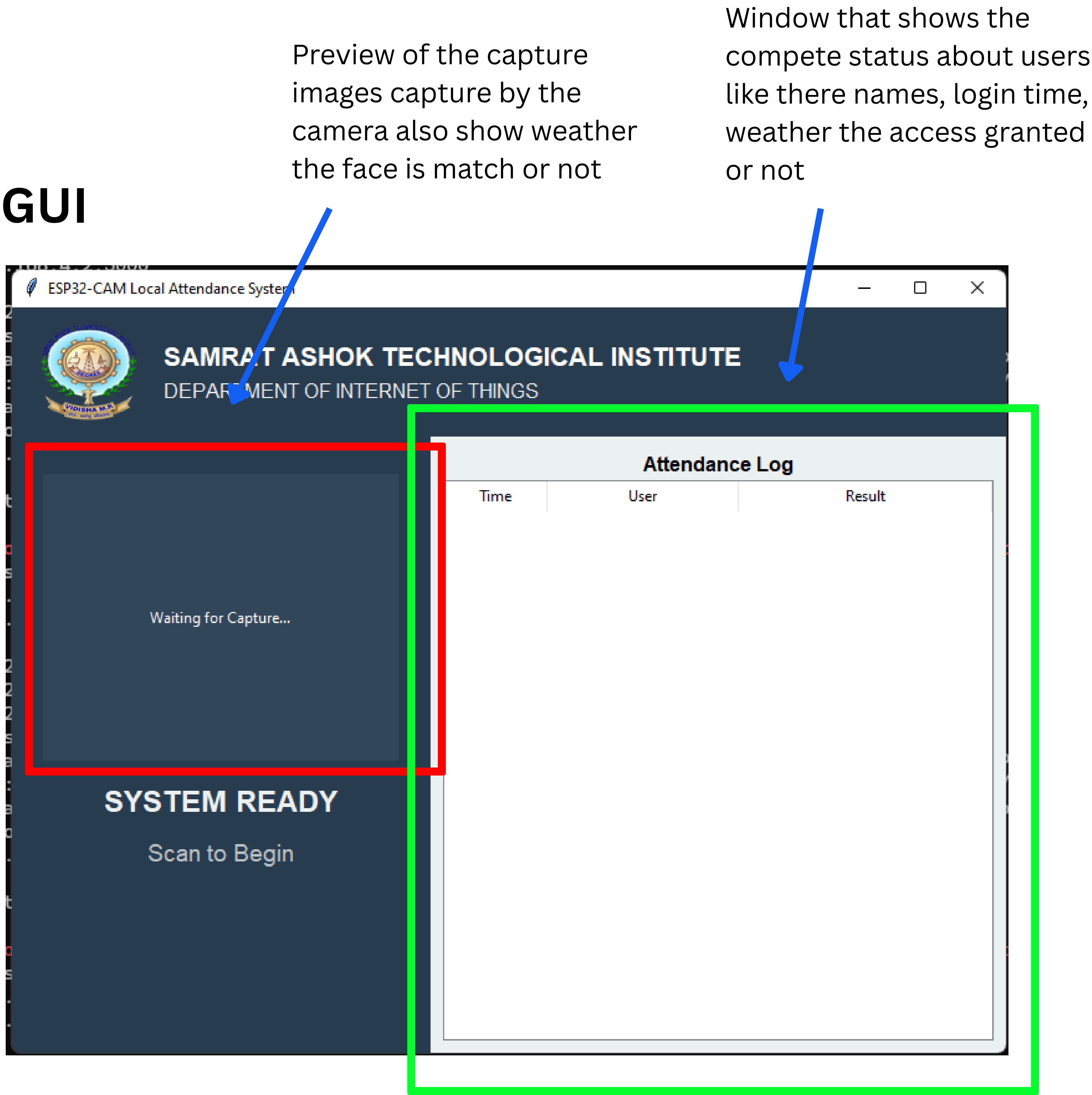
Vector conversion for Details



Pcb 3d View

Sowfare Description

GUI



Terminal

Face loaded stored in the folder name - Known_faces

Ip address and port numner on which flask server listing

camera capturing images and storing them in an forder name - capture_received

```
PS C:\Users\lnc\Documents\GitHub\Camera-based-verificaion-system-> python python_file_g
C:\Users\lnc\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\
ition_models\__init__.py:7: UserWarning: pkg_resources is deprecated as an API. See htt
_resources package is slated for removal as early as 2025-11-30. Refrain from using thi
from pkg_resources import resource_filename

[SYSTEM] Loading faces...
[SYSTEM] 1 faces loaded.
* Serving Flask app 'python_file_gui'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.4.2:5000
Press CTRL+C to quit
192.168.4.1 - - [04/May/2026 12:25:37] "POST /upload HTTP/1.1" 200 -
192.168.4.1 - - [04/May/2026 12:25:43] "POST /upload HTTP/1.1" 200 -
192.168.4.1 - - [04/May/2026 12:25:48] "POST /upload HTTP/1.1" 200 -
192.168.4.1 - - [04/May/2026 12:25:54] "POST /upload HTTP/1.1" 200 -
192.168.4.1 - - [04/May/2026 12:26:02] "POST /upload HTTP/1.1" 200 -
```

Initilizing the python file

Libraries required to run the python program

```
import os
import cv2
import csv
import numpy as np
import face_recognition
from datetime import datetime
from flask import Flask, request
from threading import Thread
import tkinter as tk
from tkinter import ttk
from PIL import Image, ImageTk
```

Defining the directories and file name that store the data such as authorized faces and captured faces

```
# Settings
KNOWN_FACES_DIR = "known_faces"
RECEIVED_DIR = "received_captures"
ATTENDANCE_FILE = "attendance.csv"

# Ensure directories exist
for folder in [KNOWN_FACES_DIR, RECEIVED_DIR]:
    if not os.path.exists(folder):
        os.makedirs(folder)

known_encodings = []
known_names = []
```

Load all the stored faces just after running the program

```
def load_faces():
    print("[SYSTEM] Loading faces...")
    for filename in os.listdir(KNOWN_FACES_DIR):
        if filename.lower().endswith((".jpg", ".png", ".jpeg")):
            path = os.path.join(KNOWN_FACES_DIR, filename)
            image = face_recognition.load_image_file(path)
            encodings = face_recognition.face_encodings(image)
            if encodings:
                known_encodings.append(encodings[0])
                known_names.append(os.path.splitext(filename)[0])
    print(f"[SYSTEM] {len(known_names)} faces loaded.")

@app.route('/upload', methods=['POST'])
```

*when the program is running and we store the new face in the known_faces folder and then we have to rerun the program to load that face *

```

def upload():
    if not request.data: return "NO DATA", 400

    nparr = np.frombuffer(request.data, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if img is None: return "INVALID", 400

    # --- FIX: Rotate image 180 degrees for upside-down camera ---
    img = cv2.rotate(img, cv2.ROTATE_180)

    # Save capture
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    save_path = os.path.join(RECEIVED_DIR, f"cap_{timestamp}.jpg")
    cv2.imwrite(save_path, img)

    # Recognition logic
    rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    face_locs = face_recognition.face_locations(rgb_img)
    face_encs = face_recognition.face_encodings(rgb_img, face_locs)

    result_text = "FACE NOT MATCHED"
    status_color = "red"
    display_name = "ACCESS DENIED"

    if face_encs:
        matches = face_recognition.compare_faces(known_encodings, face_encs[0], tolerance=0.5)
        if True in matches:
            name = known_names[matches.index(True)]
            log_attendance(name)
            result_text = "FACE MATCHED"
            display_name = f"WELCOME {name.upper()}"
            status_color = "green"

    # Update GUI from background thread
    update_gui(save_path, result_text, display_name, status_color)

    return f"{display_name} ATTENDANCE MARKED" if "WELCOME" in display_name else "ACCESS DENIED", 200

```

*Error 400 - data or incorrect formate data received
 Error 200 - data received successfully *

This section of code can communicate with the hardware (ESP-Cam) and send the data to show on display

```

def log_attendance(name):
    now = datetime.now()
    file_exists = os.path.isfile(ATTENDANCE_FILE)
    with open(ATTENDANCE_FILE, 'a', newline='') as f:
        writer = csv.writer(f)
        if not file_exists:
            writer.writerow(["Name", "Date", "Time"])
        writer.writerow([name, now.strftime("%Y-%m-%d"), now.strftime("%H:%M:%S")])

```

Mark attendace in csv file

```

# --- Frontend Logic (Tkinter) ---
def update_gui(img_path, status, name, color):
    # Update Image
    img = Image.open(img_path)
    img = img.resize((300, 225), Image.Resampling.LANCZOS)
    photo = ImageTk.PhotoImage(img)
    lbl_img.config(image=photo)
    lbl_img.image = photo

    # Update Status
    lbl_status.config(text=status, fg=color)
    lbl_name.config(text=name)

    # Update Log Table
    tree.insert("", 0, values=(datetime.now().strftime("%H:%M:%S"), name, status))

```

Update attendance in gui window

```

def start_server():
    # Use threaded=True to handle requests without blocking
    app.run(host='0.0.0.0', port=5000, threaded=True)

```

Flask server listing on port number 5000

Instructions to optrat the python program

After downloading the file form github

Store authorized faces here
with the person name

Upload on ESP-Cam

Upload on ESP8266

After connecting you
laptop with the esp cam
run this file

.git	✓	5/4/2026 9:55 AM	File folder	
esp_cam_file	✓	5/4/2026 8:37 PM	File folder	
ESP8266-Lcd-Control-Code	✓	5/4/2026 8:37 PM	File folder	
known_faces	✓	5/4/2026 9:37 AM	File folder	
college_logo	✓	5/4/2026 9:47 AM	PNG File	103 KB
esp_cam_file	✓	5/4/2026 12:09 PM	INO File	3 KB
ESP8266-Lcd-Control-Code	✓	5/4/2026 12:16 PM	INO File	3 KB
Importan_links_and_information	✓	5/4/2026 9:35 AM	Markdown Source File	3 KB
Overall_Working	✓	5/4/2026 9:35 AM	PNG File	1,117 KB
python_file.py	✓	5/4/2026 9:35 AM	Python.File	3 KB
python_file_gui.py	✓	5/4/2026 9:48 AM	Python.File	6 KB
Schematic_Esp_Cam_Attendance_system	✓	5/4/2026 9:35 AM	PNG File	61 KB

Please make to directory in same folder where the python file located with name **known_faces and **Capture_received** before run the python code*

Please change the ip address in the ESP cam code to your laptop **Ip Address and **port number***

****Please follow the instruction according to the serial number ****

